

RELAZIONE

1. INTRODUZIONE

L'obiettivo di questo progetto è realizzare una rete neurale in grado di prevedere se un cliente, contattato telefonicamente da una banca, sottoscrive con essa un deposito a termine.

Il problema, dunque, è di classificazione binaria.

2. DATASET

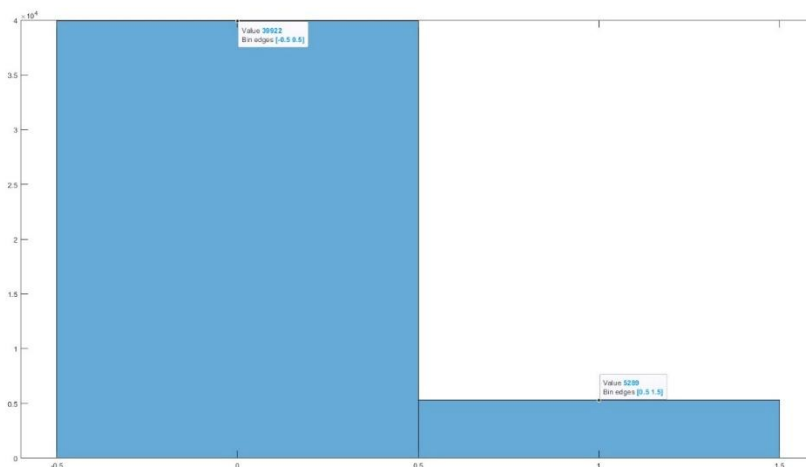
Il dataset è stato scaricato dal sito “UCI Machine Learning Repository” (<https://archive.ics.uci.edu/dataset/222/bank+marketing>).

La prima fase consiste nel processare i dati per eliminare campioni con attributi mancanti e righe che si ripetono, e nell'analizzare le distribuzioni delle feature e della label.

2.1 Analisi e codifica della label

La label è stata rappresentata utilizzando i valori “yes” e “no”, per cui è stata eseguita la conversione in formato numerico, scegliendo come convenzione di utilizzare un uno per rappresentare “yes”, e uno zero per rappresentare “no”.

Analizzando la label si ha la seguente distribuzione di 1 e 0:



Dall'immagine è possibile notare che il dataset è sbilanciato, con una presenza di 1 molto minore degli 0. Pertanto, sarà necessario fare attenzione alle metriche di validazione e di valutazione da utilizzare.

2.2 Analisi e codifica delle feature categoriche

Le feature categoriche “job”, “marital”, “education”, “default”, “housing”, “loan”, “contact”, “poutcome” sono state codificate con la codifica one-hot.

2.3 Analisi e codifica delle feature numeriche

Per codificare i giorni dell'anno in seno e coseno è necessario modificare il periodo delle due funzioni, portandolo da 2π a 365. Per fare ciò si moltiplica l'argomento di entrambe le funzioni per $\frac{2\pi}{365}$, in modo tale che $T = \frac{2\pi}{\frac{2\pi}{365}} = 365$.

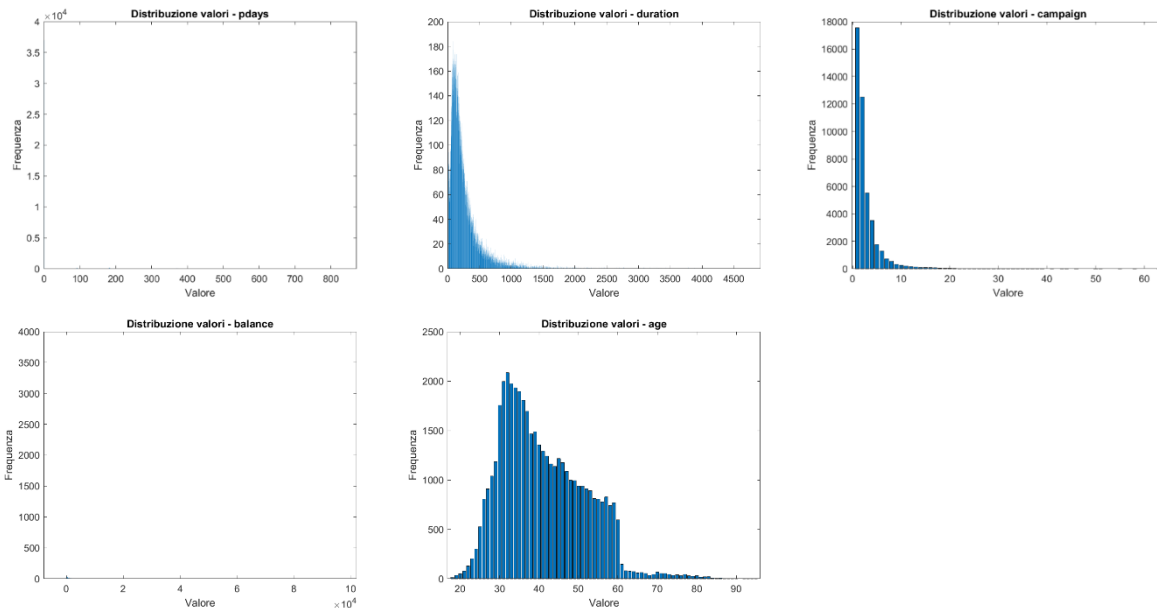
La feature “pdays” è una feature numerica, che indica quanti giorni sono passati dall'ultima volta che il cliente è stato contattato per la campagna telefonica precedente. Leggendo le descrizioni delle feature, “pdays” assume valore -1 se il cliente non è stato contattato in campagne precedenti. Dato che tale valore non è coerente con la scala utilizzata, si decide di aggiungere la colonna binaria “was_not_previously_contacted”, dove si usa 1 se “pdays” è -1, 0 altrimenti. Nella colonna “pdays” si sostituisce -1 con 0, dove il significato di 0 è determinato dalla colonna binaria aggiunta.

3. SUDDIVISIONE DATASET

Per la fase di addestramento si decide di suddividere il dataset per il 33% in test set, e la parte rimanente in training set. La suddivisione è randomica e stratificata, in modo tale che nel test set ci sia il 33% degli esempi della classe in minoranza.

4. STANDARDIZZAZIONE FEATURES NUMERICHE

Di seguito sono riportate le distribuzioni delle features numeriche.



Come è possibile notare, le features hanno ordini di grandezza differenti. Si decide quindi di standardizzarle, in modo tale da avere scale confrontabili. La formula utilizzata è

$$x_{standardized} = \frac{x - \mu}{\sigma}$$

Dove μ e σ sono, rispettivamente, la media e la varianza campionarie della feature x . Questi due valori sono stati calcolati utilizzando i campioni di addestramento, in modo tale da evitare data leakage.

5. SCELTA LOSS, TERMINE DI REGOLARIZZAZIONE E ALGORITMO DI OTTIMIZZAZIONE

5.1 Scelta loss

La loss scelta è la binary cross-entropy pesata:

$$-w_{true}y\log(\varphi) - w_{false}(1 - y)\log(1 - \varphi)$$

Dove w_{true} e w_{false} sono i pesi attribuiti a ciascuna classe. I pesi sono stati calcolati usando solo gli esempi di addestramento. Le formule utilizzate per calcolare i pesi sono:

$$w_{true} = \frac{\text{numero campioni negativi}}{\text{numero campioni positivi}}$$

$$w_{false} = \frac{\text{numero campioni positivi}}{\text{numero campioni negativi}}$$

In questo modo il modello darà maggiore peso alla classe dei positivi, che ha un numero di campioni molto minore, per cui sbagliare la predizione di un esempio positivo è più costoso che sbagliare la predizione di un esempio negativo.

Notare che in questo caso il gradiente della loss rispetto a z^{L+1} , nel caso in cui nel layer di uscita si utilizzi funzione di attivazione sigmoide, diventa:

$$-w_{true}y(1 - \varphi) + w_{false}(1 - y)\varphi$$

Il quale, tenendo conto del fatto che y può assumere valori in $\{0, 1\}$, può essere riscritta in forma compatta come:

$$w(\varphi - y)$$

5.2 Termine di regolarizzazione

In questo caso si è utilizzata la regolarizzazione l_2 .

5.3 Algoritmo di ottimizzazione

L'algoritmo di ottimizzazione utilizzato è il metodo del gradiente stocastico con momentum. Lo stepsize è costante.

6. VALIDAZIONE

6.1 Algoritmo di validazione

L'algoritmo di validazione utilizzato è la k-fold cross validation. La cross validation consente di avere una valutazione più robusta degli iperparametri scelti. Inoltre, è stato utilizzato $k = 5$, in modo tale da ridurre il tempo necessario per validare una singola architettura, e per avere dei fold con un numero non troppo basso di campioni appartenenti alla classe in minoranza. Anche in questo caso la suddivisione dei fold è stratificata e randomica.

6.2 Metriche

Le metriche adottate per questo task tengono conto dello sbilanciamento del dataset: in questo caso, infatti, l'accuratezza non è una buona metrica di valutazione, poiché un modello che predice sempre negativo avrebbe un valore della metrica pari a $\frac{39922}{39922+5289} = 0.88$. Dunque, le metriche utilizzate sono precision e recall, definite come:

$$precision = \frac{True\ Positives}{True\ Positives + False\ Positives} \quad recall = \frac{True\ Positives}{True\ Positives + False\ Negatives}$$

La precision indica la percentuale dei campioni che sono veramente positivi, su tutti i campioni che il modello ha predetto come positivi. La recall indica la capacità del modello di identificare i campioni positivi. Queste due metriche vengono utilizzate assieme perché, se considerate singolarmente, mettono in evidenza solo un aspetto del modello: ad esempio, un classificatore che predice sempre positivo ha una recall alta, ma la precision sarà uguale alla percentuale di campioni positivi nel dataset, che in questo caso sarà molto bassa.

Per analizzare le due metriche, supponendo che non identificare un campione positivo sia più costoso che identificare un campione negativo, si utilizza l’F2-score, definito come:

$$F_2 = \frac{5 \times Precision \times Recall}{4 \times Precision + Recall}$$

Questa metrica è più sensibile alle variazioni della recall.

6.3 Iperparametri

Gli iperparametri di addestramento sono:

- Stepsize;
- Momentum;
- Numero di epoche;
- Taglia dei minibatch: in questo caso si è cercato di usare una taglia abbastanza grande per includere un numero sufficiente di campioni positivi all’interno di ogni minibatch;
- Coefficiente di regolarizzazione;
- Metodo di scelta dei pesi iniziali: il metodo scelto è l’inizializzatore di He, adatto nel caso si utilizzi la ReLU come funzione di attivazione dei layer nascosti. Questa tecnica consiste nell’inizializzare le matrici dei pesi di ciascuno strato effettuando estrazioni randomiche da una distribuzione normale con media nulla e varianza uguale a $\frac{2}{fan_{in}}$, dove fan_{in} è il numero di input dello strato. Il vantaggio di questa scelta è che le varianze delle attivazioni rimangono costanti, evitando così problemi di exploding/vanishing gradient. I bias sono stati inizializzati a 0.

Gli iperparametri della rete neurale sono:

- Numero di strati nascosti;
- Funzione di attivazione degli strati: per gli strati nascosti è stata utilizzata la ReLU, mentre per lo strato di output è stata utilizzata la funzione sigmoide. Nella fase di backpropagation, la ReLU non è derivabile in 0. Quindi, si è deciso di porre il suo gradiente a 0 in corrispondenza di valori nulli. Tale scelta è motivata dal fatto che

ReLU(0) è nulla, cioè in questo caso il neurone si spegne, e non influisce in alcun modo sulla predizione finale;

- Numero di neuroni per strato: per lo strato di output è stato utilizzato un solo neurone anziché due con funzione di attivazione softmax, in modo tale da poter impostare la soglia di positività;
- Soglia di positività: indica la probabilità minima che deve essere predetta dal modello per poter classificare il campione come positivo. La soglia è stata introdotta perché il dataset è sbilanciato, e usare una soglia di 0.5 massimizzerebbe l'accuratezza, non l' F_2 -score. Nella fase di cross-validation, al termine dell'addestramento di ogni modello vengono calcolate le probabilità sul validation fold. Successivamente si sceglie la soglia che massimizza l' F_2 -score sulle predizioni dei 5 modelli. L' F_2 -score considerato per valutare la combinazione degli iperparametri è la media degli F_2 -score calcolati su ogni singolo modello, usando la soglia calcolata.

6.4 Scelta degli iperparametri

Per trovare una buona combinazione di iperparametri è stato utilizzato il seguente metodo:

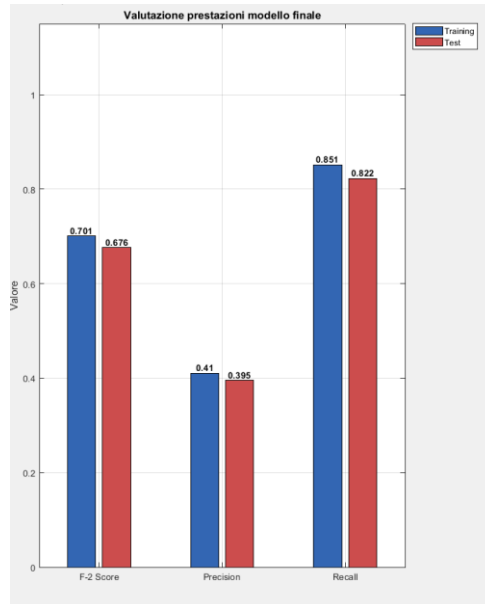
1. Sono state provate diverse architetture di reti neurali, lasciando invariati gli iperparametri di addestramento. Tali iperparametri sono stati scelti in base alla stabilità dell'addestramento dei modelli;
2. Sono state scelte le due architetture con F_2 -score più alto;
3. Entrambe le architetture sono state addestrate variando learning rate, taglia del minibatch e numero di epoche, con lo scopo di rendere l'addestramento più veloce possibile e allo stesso tempo stabile, e per convergere in punti di minimo locale di buona qualità.

Nel file validation_result.pdf, inserito in allegato assieme alla relazione e al codice sorgente, sono presenti le combinazioni di iperparametri scelte, e le relative metriche. È stata evidenziata in rosso la combinazione con le prestazioni migliori, riportata anche di seguito:

```
----- Model 2 -----  
----- Neural Network hyperparameters -----  
Network configuration (neurons and activations) = {neurons:[38 128 64 32 16 1],  
act_funcs:{relu, relu, relu, relu, sigmoid}}  
Parameter initialization method = he_normal  
Positivity threshold = 0.9  
  
----- Training hyperparameters -----  
Learning rate = 0.002  
Momentum coefficient = 0.95  
Number of epochs = 100  
Minibatch size = 128  
Regularization coefficient = 0.004  
  
----- Results -----  
Validation F2 = 0.6963  
Training F2 = 0.7027  
  
Validation Precision = 0.4010  
Training Precision = 0.4045  
  
Validation Recall = 0.8547  
Training Recall = 0.8622
```

7. MODELLO FINALE

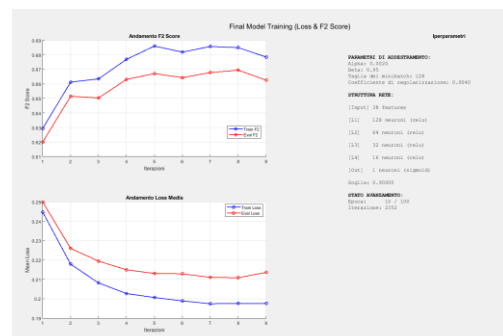
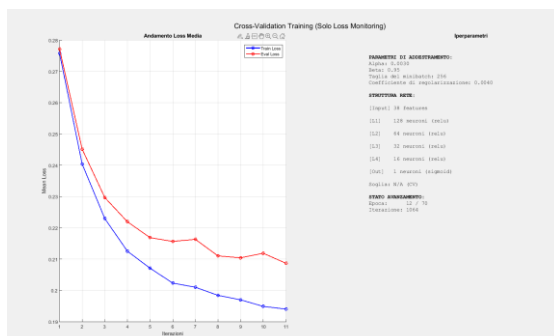
Il modello testato ha prestazioni molto simili rispetto a quelle misurate in fase di validazione:



pertanto, si decide di utilizzarlo come modello finale.

8. DASHBOARD DI ADDESTRAMENTO

Per visualizzare l'addestramento dei modelli è stata creata una dashboard. La dashboard mostra gli iperparametri di addestramento e della rete neurale, più l'andamento della loss media (senza termine di regolarizzazione) e dell' F_2 -score: quest'ultima è mostrata solo nella fase di addestramento del modello finale, poiché in fase di validazione è necessario addestrare i 5 modelli per poter calcolare la soglia di assegnamento della classe. Tali metriche sono calcolate al termine di ogni epoca per ridurre il costo computazionale dell'addestramento. Al termine dell'addestramento del modello finale sono anche mostrati gli istogrammi di F_2 -score, precision e recall. Di seguito sono riportate le immagini della dashboard mostrata in fase di validazione e di addestramento del modello finale:



9. POSSIBILI MIGLIORAMENTI

Nonostante la robustezza del modello finale, ci sono diversi punti su cui si potrebbero introdurre dei miglioramenti. Infatti, l'algoritmo di ottimizzazione utilizzato ha il vincolo di utilizzare lo stesso stepsize: un possibile miglioramento consiste nell'implementare uno scheduler del learning rate, in modo tale che nelle prime epoche si può utilizzare uno stepsize alto per superare i punti di minimo locale più vicini, per poi diminuirlo per avere convergenza. Inoltre, l'utilizzo di algoritmi di ricerca degli iperparametri di tipo stocastico consentirebbe così di provare più combinazioni in tempi ragionevoli. Infine, si potrebbe confrontare la rete neurale con altri modelli, come alberi decisionali e random forest.